

OCR A-Level Computer Science (H446) LMC Assembly Cheat Sheet

Little Man Computer = OCR's assembly language. 100 mailboxes (00–99) of 3-digit numbers (000–999). One accumulator.

The machine

- **ACC** accumulator – the only general register; all arithmetic happens here
- **PC** program counter – address of the next instruction (starts at 00)
- **MAR / MDR / CIR** – address reg. / data reg. / current instruction reg.
- **Instructions and data are both just numbers in mailboxes** (stored-program concept)
- **Machine code:** hundreds digit = opcode, last two digits = mailbox address (e.g. 306 = STA 06)

Writing assembly

[label] MNEMONIC [operand] // comment

- One instruction per line. Operand is a **label** (or a mailbox number).
- **name DAT** reserves a mailbox (starts at 0). **one DAT 1** starts at 1.
- Code is placed from mailbox 00 in source order; DAT lines take a mailbox too.
- Put every DAT AFTER HLT – otherwise the CPU executes it.
- Assembled in two passes: (1) build the symbol table, (2) generate the code.

The 11 instructions

Mnemonic	Opcode	What it does	Effect (register transfer)
ADD	1xx	Add the contents of mailbox xx to the accumulator	$ACC \leftarrow ACC + [xx]$
SUB	2xx	Subtract the contents of mailbox xx from the accumulator	$ACC \leftarrow ACC - [xx]$
STA	3xx	Store the accumulator in mailbox xx	$[xx] \leftarrow ACC$
LDA	5xx	Load the contents of mailbox xx into the accumulator	$ACC \leftarrow [xx]$
BRA	6xx	Branch always: jump to xx	$PC \leftarrow xx$
BRZ	7xx	Branch to xx if the accumulator is zero	if $ACC = 0$ then $PC \leftarrow xx$
BRP	8xx	Branch to xx if the accumulator is zero or positive	if $ACC \geq 0$ then $PC \leftarrow xx$
INP	901	Input a number into the accumulator	$ACC \leftarrow \text{input}$
OUT	902	Output the accumulator	$\text{output} \leftarrow ACC$
HLT	000	Stop the program (COB is an alias in some simulators)	stop
DAT	–	Not an instruction. Reserves a labelled mailbox, with an optional starting value (default 0)	assembler only – no opcode

Remember

- **LDA 10** loads the **contents of mailbox 10**, not the number 10. To use a constant, give it a DAT (e.g. **one DAT 1** then **ADD one**).
- There is no compare instruction: subtract, then test. **Equal?** SUB then BRZ. **Greater or equal?** SUB then BRP. There is no “branch if negative”: invert the test.
- Opcode 4xx is unused. INP, OUT, HLT take no operand. No multiply, divide, AND/OR or shifts: build them from the 11 above.

Fetch–decode–execute in the LMC

Fetch (every instruction)	Execute
MAR ← PC PC ← PC + 1 MDR ← [MAR] CIR ← MDR Decode: control unit splits CIR into opcode (first digit) and operand (last two).	LDA xx MAR ← xx ; MDR ← [MAR] ; ACC ← MDR STA xx MAR ← xx ; MDR ← ACC ; [MAR] ← MDR ADD xx MAR ← xx ; MDR ← [MAR] ; ACC ← ACC + MDR SUB xx MAR ← xx ; MDR ← [MAR] ; ACC ← ACC - MDR BRA xx PC ← xx BRZ xx if ACC = 0 then PC ← xx BRP xx if ACC ≥ 0 then PC ← xx INP ACC ← input OUT output ← ACC <i>A branch overwrites the PC after fetch has already incremented it.</i>

Program patterns

Sequence: add two numbers	Selection: if a ≥ b	Iteration
Addr Code Source 00 901 INP 01 306 STA first 02 901 INP 03 106 ADD first 04 902 OUT 05 000 HLT 06 000 first DAT <i>Inputs 5, 3 give output 8.</i>	LDA a SUB b // a - b BRP athen // a >= b ...else... BRA endif athen ...then... endif ...	// while ACC <> 0 loop BRZ end ...body... SUB one BRA loop end HLT one DAT 1 Sentinel loop: INP, BRZ finish, body, BRA loop

Pitfalls and addressing modes

Top mistakes

- Treating LDA 5 as “load the number 5”.
- Placing DAT lines before or inside the code. DAT 0 = 000 = HLT, so the program stops at once.
- Forgetting HLT: the CPU runs on into the data and executes it.
- Forgetting BRA loop at the bottom of a loop.
- Branching on stale ACC: BRZ/BRP test the ACC *now*. Check what it holds.
- Using a mnemonic (or a repeated name) as a label.
- Mixing up OCR (LMC) with AQA’s ARM-style assembly.

Addressing modes (spec concept)

Example memory: [5] = 12, [12] = 99, [7] = 40, index register IX = 2

Mode	Operand is...	LDA 5 gives...
Immediate	the value itself	5
Direct	the address of the value	[5] = 12
Indirect	the address of the address	[[5]] = [12] = 99
Indexed	a base address; add the index register	[5 + IX] = [7] = 40

The LMC itself only has **direct** addressing. Indexed access can be imitated by adding 1 to an instruction word (self-modifying code).

OCR H446 uses the LMC as its assembly language; AQA uses a different instruction set. Always check against the current specification and past papers.